Session 18

Introducing New Features of Apache Web Server 2.4

For Aptech Center Use Only

Objectives

- Explain the new features of Apache Web Server 2.4
- Explain the enhanced Multi-Processing Modules (MPMs)
- Describe how to configure Apache Web Server 2.4
- Explain the core Apache Web Server features
- Describe asynchronous support in Apache Web Server 2.4
- Explain how to reduce the memory usage

For Aptech Center Use Only

© Aptech Limited

Enhancements to MPMs in Apache Web Server 2.4

- Enhanced MPMs in Apache 2.4:
  - Run-time Loadable MPMs
  - Event MPMs



For Aptech Center Use Only

Using Run-time Loadable MPMs

- Use the `LoadModule` directive to create the run-time loadable MPMs.
- On Unix-like platforms, MPMs can be created as Dynamic Shared Object (DSO) modules and dynamically loaded to the server similar to other DSO modules.
- Syntax to use the `LoadModule` directive:

```
LoadModule <module> <filename>
```

For Aptech Center Use Only

Example: Build an MPM as DSO Module 1-4

- Create a dynamic module for building MPM by using the following code:

```
$ bin/apxs -g -n poc_mpm
```

- Figure shows output of the command:

```
$ bin/apxs -g -n poc_mpm
Creating [DIR] poc_mpm
Creating [FILE] poc_mpm/Makefile
Creating [FILE] poc_mpm/Modules.mk
Creating [FILE] poc_mpm/mod_poc_mdm.c
Creating [FILE] poc_mpm/.deps

$ cd poc_mdm
$ gvim mod_poc_mdm.c
```

Output of Building MPM

For Aptech Center Use Only

Example: Build an MPM as DSO Module 2-4

- After creating a module, build it using `make` command:

```
$ pwd
 /apptech/poc_mpm
$ make
```

- The output of the `make` command is:

```
/apptech//httpd/build/libtool --silent --mode=compile gcc -g
-O0 -pthread -DLINUX=2 -D_REENTRANT -D_GNU_SOURCE -
I/apptechhttpd/include -I. -I/apptech/httpd-
2.2.16/srclib/apr/include -I/apptech/httpd-
2.2.16/srclib/apr- util/include -I/apptech/httpd-
2.2.16/srclib/apr- util/xml/expat/lib -prefer-pic -c
mod_poc_mpm.c && touch mod_poc_mpm.slo

  /apptech/httpd/build/libtool --silent --mode=link gcc -g
 -O0 - pthread -o mod_poc_mpm.la -rpath
 /apptech/httpd/modules -module -avoid-version
 mod_poc_mpm.lo
```

For Aptech Center Use Only

Example: Build an MPM as DSO Module 3-4

- Then, you need to activate the module using following command:

```
$ ../bin/apxs -c -i mod_poc_mpm.c
$ cd ../modules/
$ ls
 httpd.exp mod_poc_mpm.so
```

- Finally, add the following command to the httpd.conf file:

```
LoadModule poc_rest_mpm modules/mod_poc_mpm.so
```

For Aptech Center Use Only

Example: Build an MPM as DSO Module 4-4

- To view the httpd.conf file, enter following commands on the command prompt:

```
$ cd ../conf/
$ gvim httpd.conf
```

- Following figure shows output of the httpd.conf file.

```
#
# Dynamic Shared Object (DSO) Support
#
# To be able to use the functionality of a module which was built as a DSO you
# have to place corresponding 'LoadModule' lines ar this location so the
# directives contained in it are actually available before they are used.
# Statically compiled modules (those listed by'httpd - 1') do not need
# to be loaded here.
#
# Example:
# LoadModule foof_module modues/mod_foo.so
#
LoadModule poc_mpm_module modules/mod_poc_mpm.so
SetHandler poc_mpm
```

For Aptech Center Use Only

Building Multiple MPMs as DSOs

- If you build more than one MPM using `LoadModule` directive, the following error message is displayed:

```
AH00534: httpd: Configuration error: More than one MPM
loaded
```

- To overcome this problem, use following options with the configure script:

--enable-mpms-shared=MPM-LIST: Enables a set of MPMs as DSO modules with one of the modules that executes at runtime using `LoadModule` directive.

The command to use this option is as follows:

```
./configure –enable-mpms-shared='mpm_prefork_module'
```

--enable-mpms-shared=all: Selects all MPMs that support DSO modules.
The command to use this option is as follows:

```
./configure –enable-mpms-shared=all
```

For Aptech Center Use Only

Using the Event MPM

Helps to process more requests simultaneously by offloading some of the processing work to the listeners' threads.

Releases worker threads to process new requests.

- Add following code to the configure script's arguments while building the httpd:

```
--with-mpm=event
```

Supports `AsyncRequestWorkerFactor` directive, which handles some connections in an asynchronous way.

For Aptech Center Use Only

Event MPM - AsyncRequestWorkerFactor Directive 1-2

- Adjusts limit of number of concurrent connections per process.
- A process accepts new connections if the existing number of working connections is less than the value, which is calculated using formula:

```
ThreadsPerChild + (AsyncRequestWorkerFactor * Number of idle workers)
```

where:

- **ThreadsPerChild**: Specifies the number of threads created by each child process.
- **AsyncRequestWorkerFactor**: Specifies the limit of concurrent connections per process.
- **Number of idle workers**: Specifies the worker threads that are available.

For Aptech Center Use Only

Event MPM - AsyncRequestWorkerFactor Directive 2-2

- Calculate estimation of maximum concurrent connections across all the processes by providing average value of idle worker threads using following formula:

```
(ThreadsPerChild + (AsyncRequestWorkerFactor * number of idle workers))
* ServerLimit
```

where,

ServerLimit: Specifies the upper limit on configurable number of processes.

- Use the following formula to calculate the maximum numbers of concurrent connections if all the worker threads are idle:

```
(AsyncRequestWorkerFactor + 1) * MaxRequestWorkers
```

For Aptech Center Use Only

© Aptech Limited

Example - Calculate Maximum Number of Concurrent Connections

```
ThreadsPerChild = 20
ServerLimit = 8
AsyncRequestWorkerFactor = 4
MaxRequestWorkers = 80
idle_workers = 8 (average for all the processes)

max_connections = (ThreadsPerChild +
(AsyncRequestWorkerFactor * idle_workers)) * ServerLimit
= (20 + (4 * 8)) * 8 = 416
```

- Here, 416 is the maximum number of concurrent connections.

For Aptech Center Use Only

Configuring the Apache Web Server 2.4

- Supports a number of MPMs that allow Apache Server to run in different modes, such as a process-based, hybrid (process and thread), or event-hybrid mode.

- Use following configuration directives:



- Per-module and Per-directory LogLevel Configurations
- Per-request Configuration Sections
- Configuration File Variables

For Aptech Center Use Only

Using the Per-module and Per-directory LogLevel Configurations

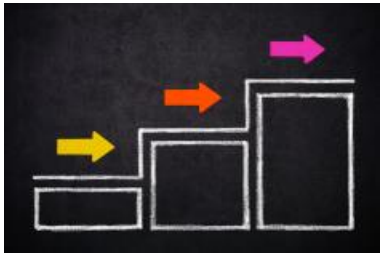
- Allows you to specify a LogLevel severity on Per-module basis.
- Helps in troubleshooting a problem with just one particular module.
- Controls the verbosity of the messages where the server logs errors by specifying the levels of the module.
- The syntax to use this directive is as follows:

```
LogLevel [module:]level [module:level]
```

For Aptech Center Use Only

Levels of the LogLevel Directive

- **emerg**: Indicates emergency situations such as the system being unusable
- **alert**: Specifies the action that should be taken immediately
- **crit**: Specifies the critical situations
- **error**: Denotes the error conditions
- **warn**: Denotes the warning conditions
- **notice**: Specifies the normal but important conditions
- **info**: Specifies the informational message about the request
- **debug**: Specifies information that helps for the debug purpose
- **trace1– trace8**: Indicates the status of the request



For Aptech Center Use Only

© Aptech Limited

Per-module LogLevel Configuration

- Specify a level with module name sets the level for that module only.
- Specify module name in three forms: module source file name, module identifier, or module identifier with the trailing `_module`.
- Use any one of the following code to specify the module name with a level:
  - `LogLevel info ssl:warn`
  - `LogLevel info mod_ssl.c:warn`
  - `LogLevel info ssl_module:warn`

For Aptech Center Use Only

Per-directory LogLevel Configuration

- Affects messages that are logged after the request has been parsed.
- Does not affect the log messages that are associated with the connection or the server.
- Change the directory for the Per-directory LogLevel configuration using the following code:

```
LogLevel info
  <Directory "/usr/local/apache/htdocs/app">
  LogLevel debug
  </Directory>
```

For Aptech Center Use Only

Using the Per-request Configuration Sections 1-3

- Sets the configuration based on per-request criteria using following sections:
  - `<If>`
  - `<ElseIf>`
  - `<Else>`
- The `<If>` section:
  - Uses in several contexts of Apache, such as .htaccess, server config, directory, and virtual host is calculated at request time to show the output of the server.
  - Contains a set of directives that execute only if a condition is satisfied and evaluates to true at runtime.
  - Syntax to use the `<If>` section is as follows:

```
<If expr> ... </If>
```

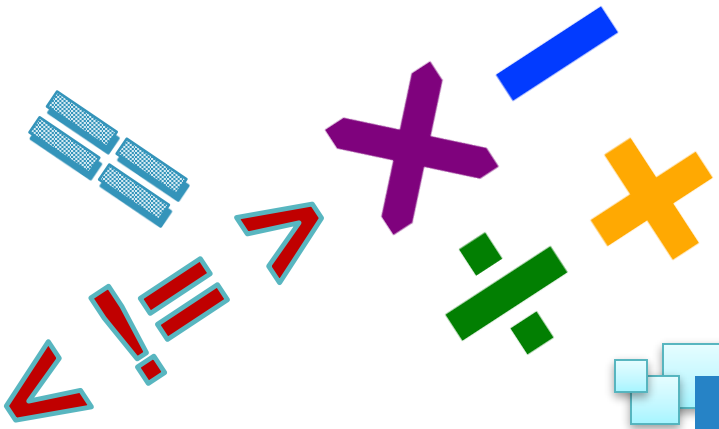
For Aptech Center Use Only

Using the Per-request Configuration Sections 2-3

- Contains several shell-like operators, such as string comparison operators (==, !=, and <), integer comparison operators (-eq and -ne), and other operators (-n, -z, -f)

- For example,

```
<If "req('Host') != 'www.rediff.com'">
    RedirectMatch (.*) http://www.rediff.com$1
  </If>
```



For Aptech Center Use Only

- The `<Elseif>` section:
  - Uses the `<ElseIf>` and `<Else>` directives if you have a set of conditions and expressions.
  - Builds multi-step `if ... elseif ... elseif ... else` logic runs for several conditions.
  - Contains directives that execute only if the condition of a last `<If>` or `<ElseIf>` section is not satisfied at runtime.
  - The syntax to use the `<ElseIf>` section is as follows:

```
<ElseIf expr> ... </ElseIf>
```

- The `<Else>` section:
  - Contains directives that execute only if the condition of a last `<If>` or `<ElseIf>` section is not fulfilled by a request at runtime.
  - The syntax to use the `<Else>` section is as follows:

```
<Else> ... </Else>
```

For Aptech Center Use Only

Using the Configuration File Variables

- The httpd.conf file is used to configure Apache Web Server.
- Use the `Define` directive to define a variable and set its value.
- Syntax to use the `Define` directive in the httpd.conf file is as follows:

```
Define variable-name [variable-value]
```



- Use the path variable in the httpd.conf file as follows:

```
ServerRoot = ${path}
DocumentRoot = ${path}
<Directory ${path}>
```

For Aptech Center Use Only

© Aptech Limited

Introducing the Core Apache HTTP Server Features

- Apache uses following core features that make a powerful and flexible Web server to work on a variety of platforms in different environments:

General-purpose Expression Parser

KeepAliveTimeout Directive

Override Configuration



For Aptech Center Use Only

Using the General-purpose Expression Parser 1-6

- Is an element of a compiler or interpreter that breaks data into smaller components for transformation into another language.
- `ap_expr` is the parser that configures all directives of the different modules of the Web Server.
- Replaces the most other expression variants in httpd.
- Uses following directives to perform the parsing function:
  - `SetEnvIfExpr`
  - `RewriteCond`
  - `<If>`

For Aptech Center Use Only

© Aptech Limited

- The `SetEnvIfExpr` Directive:
  - Sets environment variables based on an `<If>` `ap_expr` at runtime.
  - Syntax to use the `SetEnvIfExpr` directive is as follows:

```
SetEnvIfExpr expr [!]env-variable[=value]
[[!]env- variable[=value]] ...
```

For example:

```
SetEnvIfExpr "tolower(req('X-Sendfile')) ==
'd:\pics\large.iso')" iso_delivered
```

For Aptech Center Use Only

- `RewriteCond` Directive
  - Defines a rule condition and the `RewriteCond` directives come before a `REWRITERULE` directive.
  - Is used only if both the current state of the URI matches its pattern and if these conditions are met.

For Aptech Center Use Only

Syntax to use the `RewriteCond` directive is as follows:

```
RewriteCond TestString CondPattern
```

where,

- **TestString**: Specifies a string that contains the following expanded constructs with the plain text:
  - **RewriteRule backreferences**: Specifies the backreferences as $N (0<= N<= 9). $1 to $9 allow access to the assembled parts (in parentheses) of the pattern, whereas $0 allows access to the whole string matched by that pattern.
  - **RewriteCond backreferences:** Specifies the backreferences as %N (0<= N<= 9). %1 to %9 allows access to the grouped parts (in parentheses) of the pattern whereas %0 allows access to the whole string matched by that pattern.
  - **RewriteMap expansions**: Specifies the expansions as %{mapname:key|default}.
  - **Server-Variables**: Specifies the variables as %{NAME_OF_VARIABLE} where NAME_OF_VARIABLE is a string.

For Aptech Center Use Only

© Aptech Limited

List of Server Variables

HTTP headers:	connection & request:	
HTTP_ACCEPT	AUTH_TYPE	
HTTP_COOKIE	CONN_REMOTE_ADDR	
HTTP_FORWARDED	CONTEXT_PREFIX	
HTTP_HOST	CONTEXT_DOCUMENT_ROOT	
HTTP_PROXY_CONNECTION	IPV6	
HTTP_REFERER	PATH_INFO	
HTTP_USER_AGENT	QUERY_STRING	
	REMOTE_ADDR	
	REMOTE_HOST	
	REMOTE_IDENT	
	REMOTE_PORT	
	REMOTE_USER	
	REQUEST_METHOD	
	SCRIPT_FILENAME	
server internals:	**date and time:**	**specials:**
DOCUMENT_ROOT	TIME_YEAR	API_VERSION
SCRIPT_GROUP	TIME_MON	CONN_REMOTE_ADDR
SCRIPT_USER	TIME_DAY	HTTPS
SERVER_ADDR	TIME_HOUR	IS_SUBREQ
SERVER_ADMIN	TIME_MIN	REMOTE_ADDR
SERVER_NAME	TIME_SEC	REQUEST_FILENAME
SERVER_PORT	TIME_WDAY	REQUEST_SCHEME
SERVER_PROTOCOL	TIME	REQUEST_URI
SERVER_SOFTWARE		THE_REQUEST

For Aptech Center Use Only

- **CondPattern**: Specifies the condition pattern or a regular expression applied to the current instance of the `TestString` expression. `TestString` is first assessed, before being compared with `CondPattern`.

For example, if the `TestString` consists of expr, then the `CondPattern` is represented as an `ap_expr`:

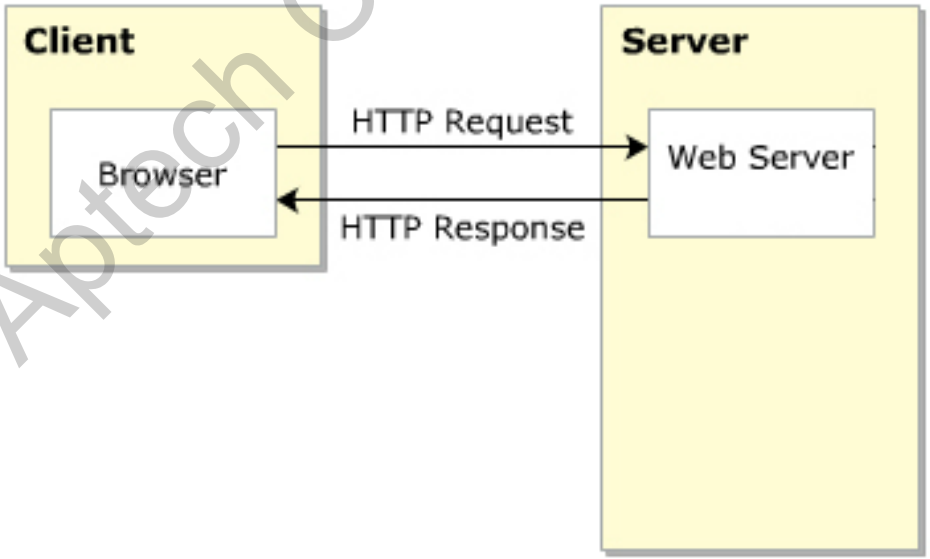
```
RewriteCond expr "! %{HTTP_REFERER} -strmatch  '*://%
{HTTP_HOST}/*'"

RewriteRule "^/images" "-" [F]
```

For Aptech Center Use Only

Using the KeepAliveTimeout Directive 1-2

- Reduces latency as well as CPU usage associated with HTTP transfers.
- Enables the `KeepAlive` directive for improving speed of Website by reusing existing connection to transfer multiple files.



For Aptech Center Use Only

Using the KeepAliveTimeout Directive 2-2

- Specifies values of the following directives to configure the `KeepAlive` directive in /etc/httpd.conf file:
  - **`KeepAlive`**: Sets the value to On.
  - **`MaxKeepAliveRequests`**: Sets the value to 500, which limits number of requests allowed on an existing connection when `KeepAlive` directive is On. The value of this directive should be set with a high value for maximum server performance.
  - **`KeepAliveTimeout`**: Sets the value to 5, which is the number of milliseconds Apache Web Server will wait for a next request before closing the connection.

For Aptech Center Use Only

© Aptech Limited

Using the Override Configuration 1-3

- Uses the `AllowOverrideList` directive to specify a list of directives allowed in the .htaccess file.
- Syntax to use `AllowOverrideList` directive is:

```
AllowOverrideList directive [directive-type] | None
```

where, `directive-type` can be one of the following groupings of directives:

- **AuthConfig**: Provides an access of authorization directives, such as `AuthName`, `AuthType`, and `Require`.
- **FileInfo**: Provides an access to the directives shown on the next slide.
- **Indexes**: Provides an access to the controlling directory indexing directives, such as `AddDescription`, `HeaderName`, and `IndexOptions`.
- **Limit**: Provides an access to the controlling host access directives, such as Allow, Deny, and Order.

For Aptech Center Use Only

© Aptech Limited

Using the Override Configuration 2-3

- Uses `FileInfo` directive to provide an access to the following directives:
  - **document types** - `ErrorDocument`, `SetHandler`, `SetInputFilter`, and so on
  - **document meta data** - `Header`, `RequestHeader`, `SetEnvIf`, and so on
  - **mod_rewrite** - `RewriteEngine`, `RewriteContd`, `RewriteRule`, and so on
  - **mod_alias** – `Redirect`, `RedirectMatch`, and so on
  - **mod_actions** – `Action`

For Aptech Center Use Only

© Aptech Limited

Using the Override Configuration 3-3

- Sets values of the AllowOverrideList and AllowOverride directives to **None**, so that the server does not read the .htaccess files and ignores them completely.
- By default, the value of the AllowOverride directive is **None** in Apache 2.4.
- For example,

```
AllowOverride None
AllowOverrideList RewriteOptions RewriteCond
```

For Aptech Center Use Only

© Aptech Limited

Summary

- Apache Web Server supports a number of Multi-Processing Modules (MPMs) that allow Apache Server to run in different modes, such as a process-based, hybrid (process and thread), or event-hybrid mode to satisfy the requirements of every infrastructure.
- You can use the LoadModule directive to create the run-time loadable MPMs.
- The event MPM helps to process more requests simultaneously by passing off some processing work to the listeners' threads, releasing the worker threads to process new requests.
- You can use the <If>, <ElseIf>, and <Else> sections to set the configuration based on per-request criteria.
- The <If> directive may be used in several contexts of Apache, such as server config, virtual host, directory, and .htaccess, and is calculated at request time to show the output of the server.

For AptechCenter Use Only

© Aptech Limited

Summary

- The <ElseIf> section contains directives that execute only if the condition of a previous <If> or <ElseIf> section is not satisfied at runtime.
- The <Else> section contains directives that execute only if the condition of a previous <If> or <ElseIf> section is not satisfied by a request at runtime.
- The httpd.conf file is used to configure Apache Web Server.
- The AllowOverrideList directive is used to specify a list of directives allowed in the .htaccess file, which is a configuration file that helps to make configuration changes of Apache Web Server on a per-directory basis.

For Aptech Center Use Only

© Aptech Limited